

Sistema di Food Delivery

Progetto di Basi di Dati

Anno Accademico 2024/2025

Indice

1	Introduzione	3
2	Requisiti della base di dati	3
3	Operazioni da effettuare sul database	3
4	Glossario dei termini	4
5	Suddivisione del testo in frasi omogenee	4
6	Diagramma Entity-Relationship	6
7	Dizionario dei dati (Entità)	6
8	Dizionario dei dati (Relationships)	7
9	Vincoli non esprimibili graficamente	8
10	Considerazioni generali	9
11	Tavola dei volumi	10
12	Operazioni di interesse	11
13	Analisi delle ridondanze	11
13.1	Ridondanza 1: Costo totale dell'ordine	11
13.2	Ridondanza 2: Tempo medio di consegna dei ristoranti	11
14	Eliminazione delle generalizzazioni	12
15	Partizionamento di entità	12
16	Eliminazione attributi multivalore	12
17	Partizionamento di relationship	12
18	Accorpamento di entità o relationship	12
19	Scelta degli identificatori	12
20	Diagramma E-R Ristrutturato	13
21	Passaggio al modello relazionale	13
22	Schema Logico	14
23	Normalizzazione	14
23.1	Prima Forma Normale (1NF)	15
23.2	Seconda Forma Normale (2NF)	15
23.3	Terza Forma Normale (3NF)	15
23.3.1	Tabella indirizzo	15
23.3.2	Tabella ordine	15
23.3.3	Altre tabelle	15

24 Progettazione fisica	15
25 Creazione del database	16
26 Realizzazione delle operazioni	18
26.1 Registrazione di un nuovo utente	18
26.2 Creazione di un nuovo ordine	19
26.3 Aggiornamento dello stato di un ordine	20
26.4 Ricerca di ristoranti per tipo di cucina	21
26.5 Visualizzazione degli ordini di un utente	21
27 Triggers	22
27.1 Trigger per notifiche automatiche su aggiornamento ordine	22
27.2 Trigger per controllo disponibilità prodotti	23
27.3 Trigger per aggiornamento automatico costo totale ordine	23
28 Viste	23
28.1 Vista ristoranti con valutazioni	23
28.2 Vista ordini completi	24
29 Conclusioni	25

1 Introduzione

Si intende realizzare una base di dati che serva da supporto a un'applicazione di food delivery. L'obiettivo del database è gestire l'intero processo di ordinazione e consegna di cibo, tenendo traccia di utenti, ristoranti, prodotti, ordini, rider e recensioni. Il sistema permetterà agli utenti di effettuare ordini da vari ristoranti, ai ristoranti di gestire i propri menu e ordini, e ai rider di gestire le consegne.

2 Requisiti della base di dati

Dell'utente si vogliono memorizzare i dati anagrafici (nome, cognome, email, password, telefono, data di nascita), la data di registrazione e lo stato dell'account. Ogni utente può avere uno o più indirizzi associati al proprio profilo.

Per quanto riguarda l'indirizzo, si memorizzano via, civico, CAP, città, provincia, nazione, e opzionalmente latitudine e longitudine per facilitare la geolocalizzazione. Ogni indirizzo può avere un'etichetta (es. "Casa", "Ufficio") e un flag che indica se è l'indirizzo predefinito per l'utente.

Del ristorante si vogliono conoscere nome, descrizione, contatti (telefono, email, sito web), orari di apertura e chiusura, giorni di apertura, tempo medio di consegna, costo di consegna base, ordine minimo, stato, data di registrazione, indirizzo e tipo di cucina. Ogni ristorante ha un proprietario che è un utente registrato nel sistema.

I prodotti offerti dai ristoranti includono nome, descrizione, prezzo, disponibilità, categoria e riferimento al ristorante che li offre.

Gli utenti possono registrare diversi metodi di pagamento, di cui si memorizza il tipo (carta di credito, PayPal, contanti, buoni pasto), il titolare, il numero mascherato per motivi di sicurezza, il token di pagamento e un flag che indica se è il metodo predefinito.

Per i rider, si memorizzano dati anagrafici, tipo di veicolo utilizzato, eventuale targa, stato di attività (disponibile, in consegna, offline) e indirizzo.

Il sistema tiene traccia degli ordini, memorizzando data, costo totale, costo di consegna, IVA applicata, note, data di consegna prevista e effettiva, utente che ha effettuato l'ordine, ristorante, indirizzo di consegna, metodo di pagamento, rider assegnato e stato dell'ordine.

Ogni ordine è composto da uno o più dettagli, che collegano i prodotti ordinati all'ordine stesso, specificando quantità, prezzo unitario ed eventuali note.

Gli utenti possono lasciare recensioni sui ristoranti e sui rider dopo aver completato un ordine. Di queste recensioni si memorizzano valutazione (da 1 a 5 stelle), commento e data.

Infine, il sistema gestisce le notifiche agli utenti, tenendo traccia di titolo, messaggio, tipo (relativo a ordine o di sistema), data di creazione, stato di lettura e data di lettura.

3 Operazioni da effettuare sul database

- Registrazione di nuovi utenti con validazione dei dati
- Aggiunta, modifica e rimozione di indirizzi degli utenti
- Inserimento e aggiornamento di ristoranti e relativi prodotti
- Creazione di nuovi ordini con selezione di prodotti, indirizzo di consegna e metodo di pagamento
- Aggiornamento dello stato degli ordini durante il loro ciclo di vita
- Assegnazione automatica o manuale dei rider agli ordini

- Registrazione di recensioni su ristoranti e rider
- Generazione di notifiche per eventi rilevanti (es. cambio stato ordine)
- Ricerca di ristoranti per tipo di cucina, valutazione o località
- Estrazione di reportistica su ordini, ristoranti più popolari e prodotti più venduti
- Gestione di utenti, permessi e ruoli (amministratore, ristoratore, cliente, rider)

4 Glossario dei termini

Termine	Definizione	Sinonimi	Collegamento
Utente	Persona registrata che utilizza il sistema per ordinare cibo	Cliente	Indirizzo, Ordine, Recensione, Metodo Pagamento
Ristorante	Esercizio commerciale che offre prodotti alimentari tramite la piattaforma	-	Prodotto, Ordine, Recensione
Prodotto	Articolo alimentare disponibile per l'ordinazione	Piatto	Ristorante, Dettaglio Ordine
Ordine	Richiesta di acquisto e consegna di uno o più prodotti	-	Utente, Ristorante, Rider, Dettaglio Ordine, Recensione
Dettaglio Ordine	Elemento di un ordine che specifica prodotto, quantità e prezzo	Voce dell'ordine	Ordine, Prodotto
Rider	Persona che effettua la consegna degli ordini	Fattorino	Ordine, Recensione
Recensione	Valutazione e commento su un ristorante o rider	Feedback	Utente, Ristorante, Rider, Ordine
Stato Ordine	Fase del ciclo di vita di un ordine	-	Ordine
Metodo Pagamento	Modalità con cui l'utente paga l'ordine	-	Utente, Ordine
Indirizzo	Localizzazione fisica per la consegna	-	Utente, Ristorante, Rider, Ordine
Notifica	Messaggio inviato all'utente per comunicare eventi rilevanti	-	Utente, Ordine

Tabella 1: Glossario dei termini

5 Suddivisione del testo in frasi omogenee

- **Frasi sugli utenti:**

"Dell'utente si vogliono memorizzare i dati anagrafici (nome, cognome, email, password, telefono, data di nascita), la data di registrazione e lo stato dell'account."

"Ogni utente può avere uno o più indirizzi associati al proprio profilo."

"Gli utenti possono registrare diversi metodi di pagamento, di cui si memorizza il tipo, il titolare, il numero mascherato, il token di pagamento e un flag che indica se è il metodo predefinito."

"Gli utenti possono lasciare recensioni sui ristoranti e sui rider dopo aver completato un ordine."

- **Frase sugli indirizzi:**

"Per quanto riguarda l'indirizzo, si memorizzano via, civico, CAP, città, provincia, nazione, e opzionalmente latitudine e longitudine per facilitare la geolocalizzazione."

"Ogni indirizzo può avere un'etichetta (es. "Casa", "Ufficio") e un flag che indica se è l'indirizzo predefinito per l'utente."

- **Frase sui ristoranti:**

"Del ristorante si vogliono conoscere nome, descrizione, contatti (telefono, email, sito web), orari di apertura e chiusura, giorni di apertura, tempo medio di consegna, costo di consegna base, ordine minimo, stato, data di registrazione, indirizzo e tipo di cucina."

"Ogni ristorante ha un proprietario che è un utente registrato nel sistema."

- **Frase sui prodotti:**

"I prodotti offerti dai ristoranti includono nome, descrizione, prezzo, disponibilità, categoria e riferimento al ristorante che li offre."

- **Frase sui rider:**

"Per i rider, si memorizzano dati anagrafici, tipo di veicolo utilizzato, eventuale targa, stato di attività (disponibile, in consegna, offline) e indirizzo."

- **Frase sugli ordini:**

"Il sistema tiene traccia degli ordini, memorizzando data, costo totale, costo di consegna, IVA applicata, note, data di consegna prevista e effettiva, utente che ha effettuato l'ordine, ristorante, indirizzo di consegna, metodo di pagamento, rider assegnato e stato dell'ordine."

- **Frase sui dettagli degli ordini:**

"Ogni ordine è composto da uno o più dettagli, che collegano i prodotti ordinati all'ordine stesso, specificando quantità, prezzo unitario ed eventuali note."

- **Frase sulle recensioni:**

"Gli utenti possono lasciare recensioni sui ristoranti e sui rider dopo aver completato un ordine. Di queste recensioni si memorizzano valutazione (da 1 a 5 stelle), commento e data."

- **Frase sulle notifiche:**

"Infine, il sistema gestisce le notifiche agli utenti, tenendo traccia di titolo, messaggio, tipo (relativo a ordine o di sistema), data di creazione, stato di lettura e data di lettura."

6 Diagramma Entity-Relationship



Figura 1: Diagramma Entity-Relationship del sistema Food Delivery

Nota: Il diagramma ER dovrebbe essere creato come immagine separata e incluso qui. Si consiglia di utilizzare strumenti come draw.io, Lucidchart o anche MySQL Workbench per creare il diagramma e poi esportarlo come immagine.

7 Dizionario dei dati (Entità)

Entità	Descrizione	Attributi	Identificatore
Utente	Utente registrato nel sistema	idUtente, nome, cognome, email, password, telefono, dataNascita, dataRegistrazione, stato	idUtente
Indirizzo	Indirizzo fisico per consegne o localizzazione	idIndirizzo, via, civico, cap, città, provincia, nazione, latitudine, longitudine, note	idIndirizzo
Ristorante	Esercizio commerciale che offre cibi	idRistorante, nome, descrizione, telefono, email, sitoWeb, orarioApertura, orarioChiusura, giorniApertura, tempoConsegnaMedio, costoConsegnaBase, ordineMinimo, stato, dataRegistrazione, tipoCucina	idRistorante
Prodotto	Articolo alimentare in vendita	idProdotto, nome, descrizione, prezzo, disponibile, categoria	idProdotto
Metodo_Pagamento	Modalità di pagamento utente	idMetodoPagamento, tipo, titolare, numeroMascherato, tokenPagamento, predefinito	idMetodoPagamento
Rider	Persona che effettua le consegne	idRider, nome, cognome, email, password, telefono, dataNascita, tipoVeicolo, targaVeicolo, statoAttività, dataRegistrazione	idRider
Stato_Ordine	Stato di avanzamento dell'ordine	idStatoOrdine, nome, descrizione	idStatoOrdine
Ordine	Richiesta di acquisto e consegna	idOrdine, dataOrdine, costoTotale, costoConsegna, ivaApplicata, note, dataConsegnaPrevista, dataConsegnaEffettiva	idOrdine
Dettaglio_Ordine	Elemento specifico di un ordine	idDettaglioOrdine, quantità, prezzoUnitario, note	idDettaglioOrdine
Recensione_Ristorante	Valutazione di un ristorante	idRecensioneRistorante, valutazione, commento, dataRecensione	idRecensioneRistorante
Recensione_Rider	Valutazione di un rider	idRecensioneRider, valutazione, commento, dataRecensione	idRecensioneRider
Notifica	Messaggio di sistema per l'utente	idNotifica, titolo, messaggio, tipo, dataCreazione, letta, dataLettura	idNotifica

8 Dizionario dei dati (Relationships)

Relazione	Descrizione	Entità coinvolte	Attributi
Associazione_Indirizzo	Associa indirizzi agli utenti	Utente, Indirizzo	etichetta, predefinito
Proprietà	Indica il proprietario di un ristorante	Utente, Ristorante	-
Ubicazione_Ristorante	Collega un ristorante al suo indirizzo	Ristorante, Indirizzo	-
Offerta	Collegamento tra ristorante e prodotti offerti	Ristorante, Prodotto	-
Possesso_Pagamento	Collega utenti ai loro metodi di pagamento	Utente, Metodo_Pagamento	-
Residenza_Rider	Collega un rider al suo indirizzo	Rider, Indirizzo	-
Effettuazione	Collega utente all'ordine effettuato	Utente, Ordine	-
Provenienza	Collega ordine al ristorante da cui proviene	Ristorante, Ordine	-
Consegna_A	Indica l'indirizzo di consegna dell'ordine	Ordine, Indirizzo	-
Pagamento	Collega ordine al metodo di pagamento usato	Ordine, Metodo_Pagamento	-
Consegna_Da	Indica il rider che effettua la consegna	Ordine, Rider	-
Stato	Indica lo stato attuale dell'ordine	Ordine, Stato_Ordine	-
Composizione	Collega ordine ai suoi dettagli	Ordine, Dettaglio_Ordine	-
Riferimento	Collega dettaglio ordine al prodotto	Dettaglio_Ordine, Prodotto	-
Valutazione_Ristorante	Collega recensione a ristorante e utente	Utente, Ristorante, Recensione_Ristorante	-
Valutazione_Rider	Collega recensione a rider e utente	Utente, Rider, Recensione_Rider	-
Ricezione	Collega utente alle notifiche ricevute	Utente, Notifica	-
Riferimento_Ordine	Collega notifica all'ordine a cui si riferisce	Notifica, Ordine	-
Riferimento_Recensione	Collega recensione all'ordine a cui si riferisce	Recensione_Ristorante/Recensione_Rider, Ordine	-

9 Vincoli non esprimibili graficamente

- I rider possono essere assegnati solo a ordini nello stato "pronto per consegna".
- Un utente può recensire un ristorante o un rider solo dopo che l'ordine è stato consegnato.
- La data di consegna effettiva di un ordine non può essere precedente alla data dell'ordine.
- La valutazione nelle recensioni deve essere compresa tra 1 e 5.
- Un utente non può avere due indirizzi con la stessa etichetta.

- Per ogni utente, può esistere al massimo un indirizzo predefinito.
- Per ogni utente, può esistere al massimo un metodo di pagamento predefinito.
- Il prezzo dei prodotti deve essere maggiore di zero.
- L'importo dell'ordine deve essere almeno pari all'ordine minimo del ristorante.

10 Considerazioni generali

Nel progettare il database per il sistema di food delivery, si è scelto di separare le recensioni per ristoranti e rider in due entità distinte per maggiore flessibilità e per consentire attributi e vincoli specifici per ciascun tipo di recensione.

Si è inoltre deciso di utilizzare un'entità separata per gli stati degli ordini anziché un semplice attributo enumerato, per facilitare l'aggiunta di nuovi stati in futuro senza modificare la struttura del database e per mantenere le descrizioni degli stati.

La relazione tra utenti e indirizzi è stata modellata come una relazione molti a molti tramite la tabella di associazione **utente_indirizzo**, permettendo così a un utente di avere più indirizzi e, teoricamente, a più utenti di condividere lo stesso indirizzo (ad esempio per famiglie o coinquilini).

Per quanto riguarda i metodi di pagamento, si è scelto di memorizzare solo informazioni parziali delle carte di credito per questioni di sicurezza, utilizzando token di pagamento forniti da servizi di pagamento di terze parti per le transazioni effettive.

Infine, il sistema di notifiche è stato progettato per essere versatile, consentendo sia notifiche legate a ordini specifici (es. "Il tuo ordine è stato consegnato") sia notifiche di sistema generiche (es. "Benvenuto nel sistema").

11 Tavola dei volumi

Concetto	Tipo	Volume
Utente	E	100.000
Indirizzo	E	150.000
Ristorante	E	2.000
Prodotto	E	100.000
Metodo_Pagamento	E	150.000
Rider	E	5.000
Stato_Ordine	E	8
Ordine	E	1.000.000
Dettaglio_Ordine	E	3.000.000
Recensione_Ristorante	E	500.000
Recensione_Rider	E	300.000
Notifica	E	5.000.000
Associazione_Indirizzo	R	150.000
Proprietà	R	2.000
Ubicazione_Ristorante	R	2.000
Offerta	R	100.000
Possesso_Pagamento	R	150.000
Residenza_Rider	R	5.000
Effettuazione	R	1.000.000
Provenienza	R	1.000.000
Consegna_A	R	1.000.000
Pagamento	R	1.000.000
Consegna_Da	R	1.000.000
Stato	R	1.000.000
Composizione	R	3.000.000
Riferimento	R	3.000.000
Valutazione_Ristorante	R	500.000
Valutazione_Rider	R	300.000
Ricezione	R	5.000.000
Riferimento_Ordine	R	3.000.000

Tabella 4: Tavola dei volumi

12 Operazioni di interesse

Operazione	Tipo	Frequenza
Registrazione di un nuovo utente	Interattiva	100/giorno
Creazione di un nuovo ordine	Interattiva	5.000/giorno
Ricerca di ristoranti per tipo di cucina o zona	Interattiva	20.000/giorno
Aggiornamento dello stato di un ordine	Interattiva	15.000/giorno
Assegnazione di un rider a un ordine	Interattiva	5.000/giorno
Inserimento di una recensione	Interattiva	2.000/giorno
Visualizzazione dello storico ordini di un utente	Interattiva	10.000/giorno
Calcolo dei ricavi giornalieri per ristorante	Batch	1/giorno
Analisi delle valutazioni dei ristoranti	Batch	1/settimana
Generazione report sui prodotti più venduti	Batch	1/settimana

Tabella 5: Operazioni di interesse

13 Analisi delle ridondanze

Nel modello concettuale sono presenti alcune possibili ridondanze che vanno analizzate per determinare se mantenerle o eliminarle.

13.1 Ridondanza 1: Costo totale dell'ordine

Il costo totale di un ordine è potenzialmente derivabile dalla somma dei prezzi unitari moltiplicati per le quantità in Dettaglio_Ordine, più il costo di consegna e l'IVA.

Analisi: Calcoliamo il numero di accessi per ottenere il costo totale di un ordine.

Con ridondanza:

- Accesso diretto al campo costoTotale in Ordine: 1 accesso

Senza ridondanza:

- Accesso a Ordine per ottenere costoConsegna e ivaApplicata: 1 accesso
- Accesso a tutti i Dettaglio_Ordine relativi all'ordine: N accessi (dove N è il numero medio di dettagli per ordine, stimato a 3)

Decisione: Mantenere la ridondanza del costoTotale, dato che l'operazione di visualizzazione degli ordini è molto frequente (10.000/giorno) e calcolare il totale ogni volta richiederebbe accessi multipli al database.

13.2 Ridondanza 2: Tempo medio di consegna dei ristoranti

Il tempo medio di consegna di un ristorante potrebbe essere calcolato sulla base dei tempi effettivi di consegna degli ordini precedenti.

Analisi: Per calcolare il tempo previsto di consegna di un nuovo ordine.

Con ridondanza:

- Accesso diretto al campo tempoConsegnaMedio in Ristorante: 1 accesso

Senza ridondanza:

- Accesso agli ultimi M ordini del ristorante: M accessi (dove M è il numero di ordini considerati per la media)
- Calcolo della media dei tempi di consegna: operazione computazionale

Decisione: Mantenere la ridondanza del tempoConsegnaMedio, poiché viene utilizzato frequentemente quando si crea un nuovo ordine (5.000/giorno) e il calcolo dinamico richiederebbe molti accessi al database.

14 Eliminazione delle generalizzazioni

Nel modello proposto non sono presenti generalizzazioni esplicite. Tuttavia, si potrebbe considerare le entità Recensione_Ristorante e Recensione_Rider come specializzazioni di un'entità generale "Recensione".

Dato che:

- Le due entità hanno attributi simili (valutazione, commento, dataRecensione)
- Ma hanno relazioni con entità diverse (Ristorante vs Rider)
- E potrebbero avere in futuro attributi specifici diversi

Si decide di mantenere le due entità separate anziché creare una generalizzazione, per maggiore flessibilità e per evitare attributi null quando non pertinenti.

15 Partizionamento di entità

Non si ritiene necessario partizionare alcuna entità, poiché ciascuna rappresenta un concetto ben definito con attributi che si riferiscono a un unico aspetto del dominio applicativo.

16 Eliminazione attributi multivalore

Non sono presenti attributi multivalore espliciti nel modello proposto. Gli attributi che concettualmente potrebbero essere multivalore (ad esempio, i numeri di telefono di un utente) sono già stati modellati come relazioni uno-a-molti o molti-a-molti (ad esempio, la relazione tra Utente e Indirizzo).

17 Partizionamento di relationship

Non si ritiene necessario partizionare alcuna relazione nel modello proposto. Le relazioni presenti sono tutte ben definite e rappresentano associazioni singole tra entità.

18 Accorpamento di entità o relationship

Non si ritiene necessario accorpare entità o relazioni, poiché il modello attuale è già ben strutturato e rappresenta correttamente i concetti del dominio.

19 Scelta degli identificatori

Per tutte le entità si utilizzano identificatori artificiali auto-incrementanti (idUtente, idRistorante, idProdotto, ecc.), che garantiscono unicità e prestazioni ottimali nelle operazioni di join tra tabelle.

20 Diagramma E-R Ristrutturato

Dopo l'analisi delle ridondanze e le altre considerazioni di progettazione, il diagramma ER rimane sostanzialmente invariato.

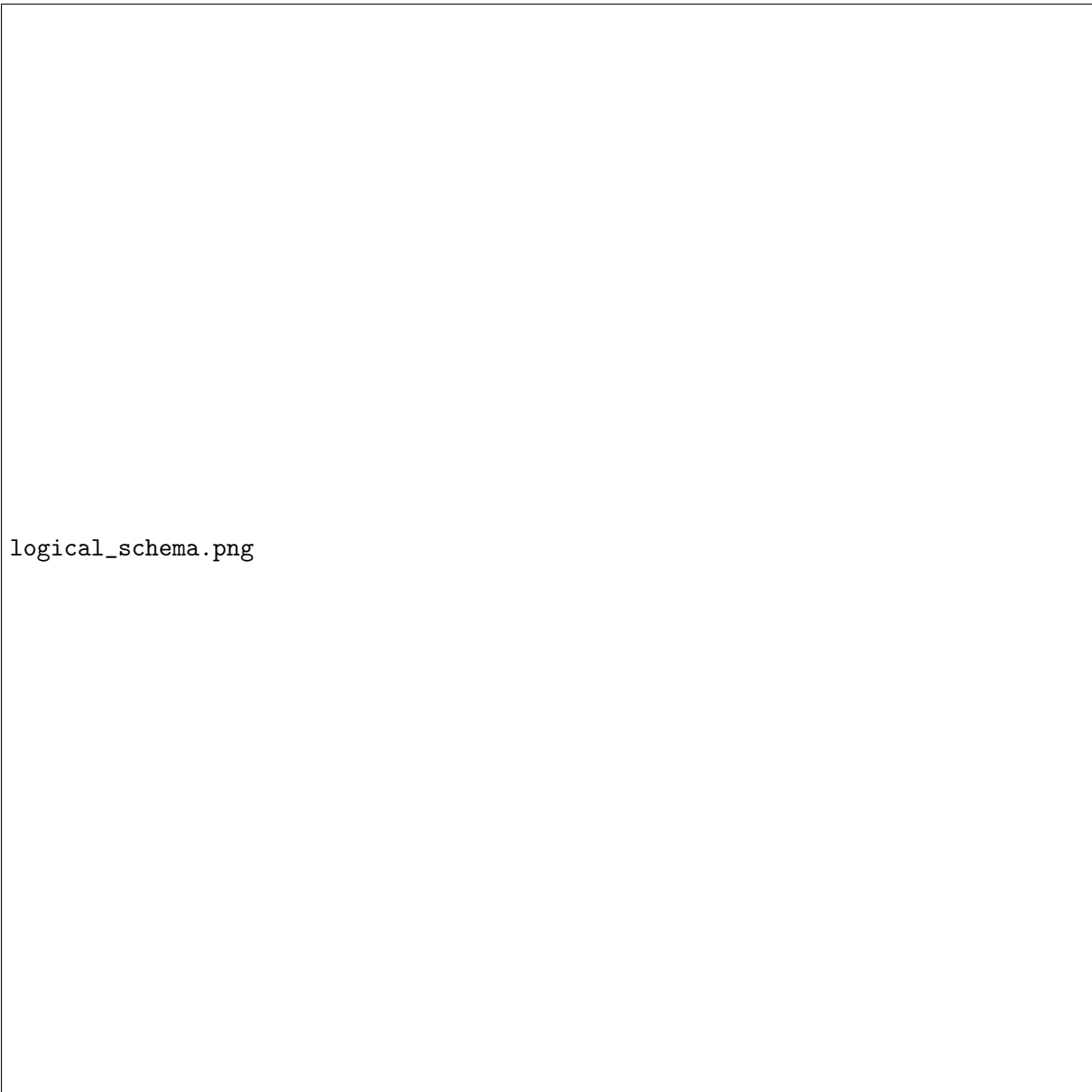
21 Passaggio al modello relazionale

Dal diagramma ER ristrutturato, si ottiene il seguente schema logico:

- **utente**(idUtente, nome, cognome, email, password, telefono, dataNascita, dataRegistrazione, stato)
- **indirizzo**(idIndirizzo, via, civico, cap, città, provincia, nazione, latitudine, longitudine, note)
- **utente_indirizzo**(idUtente, idIndirizzo, etichetta, predefinito)
FK: idUtente REFERENCES utente, idIndirizzo REFERENCES indirizzo
- **ristorante**(idRistorante, nome, descrizione, telefono, email, sitoWeb, orarioApertura, orarioChiusura, giorniApertura, tempoConsegnaMedio, costoConsegnaBase, ordineMinimo, stato, dataRegistrazione, idIndirizzo, idProprietario, tipoCucina)
FK: idIndirizzo REFERENCES indirizzo, idProprietario REFERENCES utente
- **prodotto**(idProdotto, nome, descrizione, prezzo, disponibile, categoria, idRistorante)
FK: idRistorante REFERENCES ristorante
- **metodo_pagamento**(idMetodoPagamento, tipo, titolare, numeroMascherato, tokenPagamento, predefinito, idUtente)
FK: idUtente REFERENCES utente
- **rider**(idRider, nome, cognome, email, password, telefono, dataNascita, tipoVeicolo, targaVeicolo, statoAttività, dataRegistrazione, idIndirizzo)
FK: idIndirizzo REFERENCES indirizzo
- **stato_ordine**(idStatoOrdine, nome, descrizione)
- **ordine**(idOrdine, dataOrdine, costoTotale, costoConsegna, ivaApplicata, note, dataConsegnaPrevista, dataConsegnaEffettiva, idUtente, idRistorante, idIndirizzoConsegna, idMetodoPagamento, idRider, idStatoOrdine)
FK: idUtente REFERENCES utente, idRistorante REFERENCES ristorante, idIndirizzoConsegna REFERENCES indirizzo, idMetodoPagamento REFERENCES metodo_pagamento, idRider REFERENCES rider, idStatoOrdine REFERENCES stato_ordine
- **dettaglio_ordine**(idDettaglioOrdine, quantita, prezzoUnitario, note, idOrdine, idProdotto)
FK: idOrdine REFERENCES ordine, idProdotto REFERENCES prodotto
- **recensione_ristorante**(idRecensioneRistorante, valutazione, commento, dataRecensione, idUtente, idRistorante, idOrdine)
FK: idUtente REFERENCES utente, idRistorante REFERENCES ristorante, idOrdine REFERENCES ordine
- **recensione_rider**(idRecensioneRider, valutazione, commento, dataRecensione, idUtente, idRider, idOrdine)
FK: idUtente REFERENCES utente, idRider REFERENCES rider, idOrdine REFERENCES ordine

- **notifica**(idNotifica, titolo, messaggio, tipo, dataCreazione, letta, dataLettura, idUtente, idOrdine)
FK: idUtente REFERENCES utente, idOrdine REFERENCES ordine

22 Schema Logico



logical_schema.png

Figura 2: Schema logico del database Food Delivery

Nota: Lo schema logico dovrebbe essere creato come immagine separata e incluso qui. Si consiglia di utilizzare strumenti come MySQL Workbench per generare il diagramma e poi esportarlo come immagine.

23 Normalizzazione

Analizziamo le tabelle per verificare che rispettino le forme normali.

23.1 Prima Forma Normale (1NF)

Tutte le tabelle rispettano la 1NF poiché:

- Ogni colonna contiene valori atomici
- Non ci sono gruppi di valori ripetuti
- Ogni tabella ha una chiave primaria identificata

23.2 Seconda Forma Normale (2NF)

Tutte le tabelle rispettano la 2NF poiché:

- Sono già in 1NF
- Tutti gli attributi non-chiave dipendono dall'intera chiave primaria

In particolare, per le tabelle con chiave composta come `utente_indirizzo`, gli attributi `etichetta` e `predefinito` dipendono dalla combinazione di `idUtente` e `idIndirizzo`.

23.3 Terza Forma Normale (3NF)

Analizziamo alcune tabelle per verificare che rispettino la 3NF:

23.3.1 Tabella indirizzo

Nella tabella `indirizzo`, c'è una dipendenza transitiva potenziale: il `cap` spesso dipende da `città` e `provincia`. Tuttavia, ci sono eccezioni (una città può avere più CAP) e il CAP è necessario per indirizzare correttamente. Si decide di mantenere questa struttura per semplicità e per evitare join costosi quando si accede agli indirizzi completi.

23.3.2 Tabella ordine

Nella tabella `ordine`, il campo `costoTotale` è derivabile dagli altri dati (dettagli dell'ordine, costo consegna, IVA), ma si decide di mantenerlo come ridondanza controllata per motivi di efficienza, come spiegato nell'analisi delle ridondanze.

23.3.3 Altre tabelle

Le altre tabelle sono in 3NF poiché non presentano dipendenze transitive evidenti.

24 Progettazione fisica

Per ottimizzare le prestazioni del database, si propongono i seguenti indici:

- **Indice su `utente.email` e `utente.telefono`:** Facilita le ricerche dei clienti per contatti e verifica l'unicità dei valori.
- **Indice su `ristorante.tipoCucina`:** Ottimizza le ricerche di ristoranti per tipo di cucina, una delle operazioni più frequenti.
- **Indice su `prodotto.categoria` e `prodotto.idRistorante`:** Accelera la ricerca di prodotti per categoria all'interno di un ristorante.
- **Indice su `ordine.idUtente`:** Ottimizza la ricerca degli ordini di un utente specifico.

- **Indice su ordine.idRistorante e ordine.dataOrdine:** Facilita l'analisi degli ordini di un ristorante in un periodo specifico.
- **Indice su rider.statoAttivita:** Accelera la ricerca di rider disponibili per le consegne.
- **Indice su recensione_ristorante.idRistorante:** Ottimizza il calcolo delle valutazioni medie dei ristoranti.

Inoltre, si definiscono le seguenti strategie di ottimizzazione:

- **Partizionamento della tabella ordine:** Data la grande quantità di ordini previsti, si considera un partizionamento per range sulla data dell'ordine, per migliorare le prestazioni delle query sui dati recenti.
- **Denormalizzazione strategica:** Si mantengono alcune ridondanze controllate, come il costoTotale negli ordini, per ridurre i join nelle query più frequenti.
- **Caching delle query frequenti:** Implementare meccanismi di caching per le query più frequenti, come la ricerca di ristoranti per zona o tipo di cucina.

25 Creazione del database

Di seguito riportiamo le principali istruzioni CREATE TABLE, evidenziando gli aspetti più significativi dello schema.

Listing 1: Creazione delle principali tabelle

```
-- Tabella utente
CREATE TABLE IF NOT EXISTS 'food_delivery_db'.'utente' (
  'idUtente' INT NOT NULL AUTO_INCREMENT,
  'nome' VARCHAR(45) NOT NULL,
  'cognome' VARCHAR(45) NOT NULL,
  'email' VARCHAR(100) NOT NULL,
  'password' VARCHAR(255) NOT NULL,
  'telefono' VARCHAR(20) NOT NULL,
  'dataNascita' DATE NULL,
  'dataRegistrazione' DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  'stato' ENUM('attivo', 'sospeso', 'disattivato') NOT NULL DEFAULT '
    attivo',
  PRIMARY KEY ('idUtente'),
  UNIQUE INDEX 'email_UNIQUE' (('email' ASC) VISIBLE,
  UNIQUE INDEX 'telefono_UNIQUE' (('telefono' ASC) VISIBLE)
ENGINE = InnoDB;

-- Tabella ristorante
CREATE TABLE IF NOT EXISTS 'food_delivery_db'.'ristorante' (
  'idRistorante' INT NOT NULL AUTO_INCREMENT,
  'nome' VARCHAR(100) NOT NULL,
  'descrizione' TEXT NULL,
  'telefono' VARCHAR(20) NOT NULL,
  'email' VARCHAR(100) NOT NULL,
  'sitoWeb' VARCHAR(100) NULL,
  'orarioApertura' TIME NOT NULL,
  'orarioChiusura' TIME NOT NULL,
  'giorniApertura' VARCHAR(45) NOT NULL DEFAULT 'Lun-Dom',
  'tempoConsegnaMedio' INT NOT NULL DEFAULT 30,
  'costoConsegnaBase' DECIMAL(5,2) NOT NULL DEFAULT 2.50,
  'ordineMinimo' DECIMAL(5,2) NOT NULL DEFAULT 10.00,
```

```

'stato' ENUM('attivo', 'sospeso', 'chiuso') NOT NULL DEFAULT 'attivo',
'dataRegistrazione' DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
'idIndirizzo' INT NOT NULL,
'idProprietario' INT NOT NULL,
'tipoCucina' VARCHAR(100) NULL,
PRIMARY KEY ('idRistorante'),
UNIQUE INDEX 'email_UNIQUE' ('email' ASC) VISIBLE,
INDEX 'fk_ristorante_indirizzo1_idx' ('idIndirizzo' ASC) VISIBLE,
INDEX 'fk_ristorante_utente1_idx' ('idProprietario' ASC) VISIBLE,
CONSTRAINT 'fk_ristorante_indirizzo1'
  FOREIGN KEY ('idIndirizzo')
    REFERENCES 'food_delivery_db'.'indirizzo' ('idIndirizzo')
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
CONSTRAINT 'fk_ristorante_utente1'
  FOREIGN KEY ('idProprietario')
    REFERENCES 'food_delivery_db'.'utente' ('idUtente')
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

-- Tabella ordine
CREATE TABLE IF NOT EXISTS 'food_delivery_db'.'ordine' (
  'idOrdine' INT NOT NULL AUTO_INCREMENT,
  'dataOrdine' DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  'costoTotale' DECIMAL(8,2) NOT NULL,
  'costoConsegna' DECIMAL(5,2) NOT NULL,
  'ivaApplicata' DECIMAL(4,2) NOT NULL DEFAULT 10.00,
  'note' TEXT NULL,
  'dataConsegnaPrevista' DATETIME NULL,
  'dataConsegnaEffettiva' DATETIME NULL,
  'idUtente' INT NOT NULL,
  'idRistorante' INT NOT NULL,
  'idIndirizzoConsegna' INT NOT NULL,
  'idMetodoPagamento' INT NOT NULL,
  'idRider' INT NULL,
  'idStatoOrdine' INT NOT NULL,
  PRIMARY KEY ('idOrdine'),
  INDEX 'fk_ordine_utente1_idx' ('idUtente' ASC) VISIBLE,
  INDEX 'fk_ordine_ristorante1_idx' ('idRistorante' ASC) VISIBLE,
  INDEX 'fk_ordine_indirizzo1_idx' ('idIndirizzoConsegna' ASC) VISIBLE,
  INDEX 'fk_ordine_metodo_pagamento1_idx' ('idMetodoPagamento' ASC)
    VISIBLE,
  INDEX 'fk_ordine_rider1_idx' ('idRider' ASC) VISIBLE,
  INDEX 'fk_ordine_stato_ordine1_idx' ('idStatoOrdine' ASC) VISIBLE,
  CONSTRAINT 'fk_ordine_utente1'
    FOREIGN KEY ('idUtente')
      REFERENCES 'food_delivery_db'.'utente' ('idUtente')
      ON DELETE NO ACTION
      ON UPDATE NO ACTION,
  CONSTRAINT 'fk_ordine_ristorante1'
    FOREIGN KEY ('idRistorante')
      REFERENCES 'food_delivery_db'.'ristorante' ('idRistorante')
      ON DELETE NO ACTION
      ON UPDATE NO ACTION,
  CONSTRAINT 'fk_ordine_indirizzo1'
    FOREIGN KEY ('idIndirizzoConsegna')
      REFERENCES 'food_delivery_db'.'indirizzo' ('idIndirizzo')

```

```

ON DELETE NO ACTION
ON UPDATE NO ACTION,
CONSTRAINT 'fk_ordine_metodo_pagamento1'
FOREIGN KEY ('idMetodoPagamento')
REFERENCES 'food_delivery_db'.'metodo_pagamento' ('idMetodoPagamento')
ON DELETE NO ACTION
ON UPDATE NO ACTION,
CONSTRAINT 'fk_ordine_rider1'
FOREIGN KEY ('idRider')
REFERENCES 'food_delivery_db'.'rider' ('idRider')
ON DELETE NO ACTION
ON UPDATE NO ACTION,
CONSTRAINT 'fk_ordine_stato_ordine1'
FOREIGN KEY ('idStatoOrdine')
REFERENCES 'food_delivery_db'.'stato_ordine' ('idStatoOrdine')
ON DELETE NO ACTION
ON UPDATE NO ACTION)
ENGINE = InnoDB;

```

26 Realizzazione delle operazioni

In questa sezione presentiamo le implementazioni SQL per alcune delle operazioni più significative.

26.1 Registrazione di un nuovo utente

Listing 2: Procedura per registrare un nuovo utente

```

DELIMITER //
CREATE PROCEDURE 'food_delivery_db'.'registra_nuovo_utente' (
    IN p_nome VARCHAR(45),
    IN p_cognome VARCHAR(45),
    IN p_email VARCHAR(100),
    IN p_password VARCHAR(45),
    IN p_telefono VARCHAR(20),
    IN p_dataNascita DATE,
    OUT p_idUtente INT,
    OUT p_messaggio VARCHAR(255)
)
BEGIN
    DECLARE v_email_exists INT DEFAULT 0;
    DECLARE v_telefono_exists INT DEFAULT 0;
    DECLARE v_errore_generico CONDITION FOR SQLSTATE '45000';

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        -- Gestisce gli errori SQL
        GET DIAGNOSTICS CONDITION 1
        @sqlstate = RETURNED_SQLSTATE,
        @errno = MYSQL_ERRNO,
        @text = MESSAGE_TEXT;

        SET p_messaggio = CONCAT('Errore SQL: ', @errno, ' - ', @text);
        SET p_idUtente = 0;
        ROLLBACK;
    END;

```

```

END;

-- Convalida dei dati di input
IF p_nome IS NULL OR p_nome = '' THEN
    SIGNAL v_errore_generico SET MESSAGE_TEXT = 'Il nome
        obbligatorio';
END IF;

-- Altre validazioni...

-- Verifica che email e telefono non siano gi in uso
SELECT COUNT(*) INTO v_email_exists
FROM 'food_delivery_db'.'utente'
WHERE email = p_email;

IF v_email_exists > 0 THEN
    SIGNAL v_errore_generico SET MESSAGE_TEXT = 'Email gi in uso';
END IF;

-- Verifica telefono...

-- Inizia la transazione
START TRANSACTION;

-- Inserisci il nuovo utente
INSERT INTO 'food_delivery_db'.'utente'
    (nome, cognome, email, password, telefono, dataNascita,
     dataRegistrazione, stato)
VALUES
    (p_nome, p_cognome, p_email, SHA2(p_password, 256), p_telefono,
     p_dataNascita, NOW(), 'attivo');

SET p_idUtente = LAST_INSERT_ID();

-- Crea una notifica di benvenuto
INSERT INTO 'food_delivery_db'.'notifica'
    (titolo, messaggio, tipo, idUtente, idOrdine)
VALUES
    ('Benvenuto su Food Delivery',
     CONCAT('Benvenuto ', p_nome, '! Inizia a ordinare i tuoi piatti
         preferiti.'),
     'sistema',
     p_idUtente,
     NULL);

COMMIT;

SET p_messaggio = CONCAT('Utente registrato con successo. ID: ',
    p_idUtente);
END //
DELIMITER ;

```

26.2 Creazione di un nuovo ordine

Listing 3: Procedura per creare un nuovo ordine

```

DELIMITER //

```

```

CREATE PROCEDURE 'food_delivery_db'.'.crea_nuovo_ordine' (
    IN p_idUtente INT,
    IN p_idRistorante INT,
    IN p_idIndirizzoConsegna INT,
    IN p_idMetodoPagamento INT,
    OUT p_idOrdine INT
)
BEGIN
    DECLARE v_costo_consegna DECIMAL(5,2);

    START TRANSACTION;

    -- Ottieni il costo di consegna dal ristorante
    SELECT costoConsegnaBase INTO v_costo_consegna
    FROM 'food_delivery_db'.'.ristorante'
    WHERE idRistorante = p_idRistorante;

    -- Inserisci il nuovo ordine
    INSERT INTO 'food_delivery_db'.'.ordine'
        (dataOrdine, costoTotale, costoConsegna, ivaApplicata,
         idUtente, idRistorante, idIndirizzoConsegna, idMetodoPagamento,
         idStatoOrdine)
    VALUES
        (NOW(), v_costo_consegna, v_costo_consegna, 10.00,
         p_idUtente, p_idRistorante, p_idIndirizzoConsegna,
         p_idMetodoPagamento, 1);

    -- Salva l'ID dell'ordine creato
    SET p_idOrdine = LAST_INSERT_ID();

    COMMIT;
END //
DELIMITER ;

```

26.3 Aggiornamento dello stato di un ordine

Listing 4: Procedura per aggiornare lo stato di un ordine

```

DELIMITER //
CREATE PROCEDURE 'food_delivery_db'.'.aggiorna_stato_ordine' (
    IN p_idOrdine INT,
    IN p_nuovo_stato VARCHAR(45),
    OUT p_result VARCHAR(100)
)
BEGIN
    DECLARE v_id_stato_nuovo INT;

    START TRANSACTION;

    -- Ottieni l'id del nuovo stato
    SELECT idStatoOrdine INTO v_id_stato_nuovo
    FROM 'food_delivery_db'.'.stato_ordine'
    WHERE nome = p_nuovo_stato;

    -- Aggiorna lo stato dell'ordine
    UPDATE 'food_delivery_db'.'.ordine'
    SET idStatoOrdine = v_id_stato_nuovo

```

```

WHERE idOrdine = p_idOrdine;

SET p_result = CONCAT('Stato dell\'ordine #', p_idOrdine, '
    aggiornato a ', p_nuovo_stato);
COMMIT;
END //
DELIMITER ;

```

26.4 Ricerca di ristoranti per tipo di cucina

Listing 5: Query per ricercare ristoranti per tipo di cucina

```

-- Query per trovare ristoranti con un determinato tipo di cucina e le
  loro valutazioni
SELECT
    r.idRistorante,
    r.nome,
    r.tipoCucina,
    r.orarioApertura,
    r.orarioChiusura,
    r.costoConsegnaBase,
    r.ordineMinimo,
    i.citta,
    i.provincia,
    AVG(rr.valutazione) AS valutazione_media,
    COUNT(rr.idRecensioneRistorante) AS numero_recensioni
FROM
    'food_delivery_db'.'ristorante' r
JOIN
    'food_delivery_db'.'indirizzo' i ON r.idIndirizzo = i.idIndirizzo
LEFT JOIN
    'food_delivery_db'.'recensione_ristorante' rr ON r.idRistorante = rr
    .idRistorante
WHERE
    r.stato = 'attivo'
    AND r.tipoCucina LIKE '%Italiana%'
GROUP BY
    r.idRistorante, r.nome, r.tipoCucina, r.orarioApertura, r.
    orarioChiusura,
    r.costoConsegnaBase, r.ordineMinimo, i.citta, i.provincia
ORDER BY
    valutazione_media DESC, numero_recensioni DESC;

```

26.5 Visualizzazione degli ordini di un utente

Listing 6: Query per visualizzare gli ordini di un utente

```

-- Query per visualizzare gli ordini di un utente con dettagli
SELECT
    o.idOrdine,
    o.dataOrdine,
    r.nome AS nome_ristorante,
    o.costoTotale,
    so.nome AS stato_ordine,
    GROUP_CONCAT(CONCAT(p.nome, ' (', d.quantita, ')') SEPARATOR ', ')
    AS prodotti,

```

```

        o.dataConsegnaPrevista,
        o.dataConsegnaEffettiva
FROM
    'food_delivery_db'.'ordine' o
JOIN
    'food_delivery_db'.'ristorante' r ON o.idRistorante = r.idRistorante
JOIN
    'food_delivery_db'.'stato_ordine' so ON o.idStatoOrdine = so.
        idStatoOrdine
JOIN
    'food_delivery_db'.'dettaglio_ordine' d ON o.idOrdine = d.idOrdine
JOIN
    'food_delivery_db'.'prodotto' p ON d.idProdotto = p.idProdotto
WHERE
    o.idUtente = 1 -- Sostituire con parametro
GROUP BY
    o.idOrdine, o.dataOrdine, nome_ristorante, o.costoTotale,
        stato_ordine,
    o.dataConsegnaPrevista, o.dataConsegnaEffettiva
ORDER BY
    o.dataOrdine DESC;

```

27 Triggers

Per mantenere l'integrità dei dati e automatizzare alcune operazioni, sono stati implementati diversi trigger:

27.1 Trigger per notifiche automatiche su aggiornamento ordine

Listing 7: Trigger per inviare notifiche automatiche quando un ordine viene aggiornato

```

DELIMITER //
CREATE TRIGGER after_ordine_update
AFTER UPDATE ON 'food_delivery_db'.'ordine'
FOR EACH ROW
BEGIN
    DECLARE stato_nome VARCHAR(45);

    -- Ottieni il nome dello stato
    SELECT nome INTO stato_nome FROM 'food_delivery_db'.'stato_ordine'
        WHERE idStatoOrdine = NEW.idStatoOrdine;

    -- Se lo stato cambiato, crea una notifica
    IF OLD.idStatoOrdine <> NEW.idStatoOrdine THEN
        INSERT INTO 'food_delivery_db'.'notifica'
            ('titolo', 'messaggio', 'tipo', 'idUtente', 'idOrdine')
        VALUES
            (CONCAT('Aggiornamento Ordine #', NEW.idOrdine),
                CONCAT('Il tuo ordine ora ', stato_nome),
                'ordine',
                NEW.idUtente,
                NEW.idOrdine);
    END IF;
END //
DELIMITER ;

```

27.2 Trigger per controllo disponibilità prodotti

Listing 8: Trigger per controllare la disponibilità dei prodotti prima di inserire un dettaglio ordine

```
DELIMITER //
CREATE TRIGGER before_dettaglio_ordine_insert
BEFORE INSERT ON 'food_delivery_db'.'dettaglio_ordine'
FOR EACH ROW
BEGIN
    DECLARE is_available TINYINT;

    -- Verifica se il prodotto è disponibile
    SELECT disponibile INTO is_available
    FROM 'food_delivery_db'.'prodotto'
    WHERE idProdotto = NEW.idProdotto;

    -- Se il prodotto non è disponibile, genera un errore
    IF is_available = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Il prodotto non è disponibile al momento';
    END IF;
END //
DELIMITER ;
```

27.3 Trigger per aggiornamento automatico costo totale ordine

Listing 9: Trigger per aggiornare il totale dell'ordine dopo l'inserimento di un nuovo dettaglio

```
DELIMITER //
CREATE TRIGGER after_dettaglio_ordine_insert
AFTER INSERT ON 'food_delivery_db'.'dettaglio_ordine'
FOR EACH ROW
BEGIN
    DECLARE subtotale DECIMAL(8,2);

    -- Calcola il subtotale di questo dettaglio
    SET subtotale = NEW.quantita * NEW.prezzoUnitario;

    -- Aggiorna il totale dell'ordine
    UPDATE 'food_delivery_db'.'ordine'
    SET costoTotale = costoTotale + subtotale
    WHERE idOrdine = NEW.idOrdine;
END //
DELIMITER ;
```

28 Viste

Per facilitare l'accesso ai dati più frequentemente richiesti, sono state create diverse viste:

28.1 Vista ristoranti con valutazioni

Listing 10: Vista per visualizzare i ristoranti con le loro valutazioni medie

```
CREATE VIEW 'food_delivery_db'.'vista_ristoranti_valutazioni' AS
SELECT
```



```

        r.idRistorante,
        r.nome,
        r.tipoCucina,
        r.telefono,
        i.citta,
        i.provincia,
        r.orarioApertura,
        r.orarioChiusura,
        r.giorniApertura,
        r.costoConsegnaBase,
        r.ordineMinimo,
        COALESCE(AVG(rr.valutazione), 0) AS valutazione_media,
        COUNT(rr.idRecensioneRistorante) AS numero_recensioni
FROM
    'food_delivery_db'.'ristorante' r
LEFT JOIN
    'food_delivery_db'.'recensione_ristorante' rr ON r.idRistorante = rr
        .idRistorante
JOIN
    'food_delivery_db'.'indirizzo' i ON r.idIndirizzo = i.idIndirizzo
WHERE
    r.stato = 'attivo'
GROUP BY
    r.idRistorante, r.nome, r.tipoCucina, r.telefono, i.citta, i.
        provincia,
    r.orarioApertura, r.orarioChiusura, r.giorniApertura, r.
        costoConsegnaBase, r.ordineMinimo;

```

28.2 Vista ordini completi

Listing 11: Vista per visualizzare gli ordini completi con dettagli

```

CREATE VIEW 'food_delivery_db'.'vista_ordini_completi' AS
SELECT
    o.idOrdine,
    o.dataOrdine,
    o.costoTotale,
    o.costoConsegna,
    o.ivaApplicata,
    s.nome AS stato_ordine,
    u.nome AS nome_utente,
    u.cognome AS cognome_utente,
    u.email AS email_utente,
    r.nome AS nome_ristorante,
    r.telefono AS telefono_ristorante,
    i.via,
    i.civico,
    i.citta,
    i.provincia,
    i.cap,
    CONCAT(rd.nome, ' ', rd.cognome) AS nome_rider,
    rd.telefono AS telefono_rider,
    o.dataConsegnaPrevista,
    o.dataConsegnaEffettiva
FROM
    'food_delivery_db'.'ordine' o
JOIN

```

```

        'food_delivery_db'.'.utente' u ON o.idUtente = u.idUtente
JOIN
        'food_delivery_db'.'.ristorante' r ON o.idRistorante = r.idRistorante
JOIN
        'food_delivery_db'.'.indirizzo' i ON o.idIndirizzoConsegna = i.
        idIndirizzo
JOIN
        'food_delivery_db'.'.stato_ordine' s ON o.idStatoOrdine = s.
        idStatoOrdine
LEFT JOIN
        'food_delivery_db'.'.rider' rd ON o.idRider = rd.idRider;

```

29 Conclusioni

Il database progettato fornisce una solida base per un sistema di food delivery completo e scalabile. La struttura relazionale è stata disegnata per garantire l'integrità dei dati, ottimizzare le prestazioni per le operazioni più frequenti e facilitare l'estensibilità futura.

I punti di forza del sistema includono:

- Una chiara separazione delle responsabilità tra le diverse entità del dominio
- Una gestione robusta degli ordini con un sistema di stati ben definito
- Meccanismi automatici per garantire l'integrità dei dati tramite trigger
- Procedure ottimizzate per le operazioni più frequenti
- Viste preconfigurate per le query più complesse e ripetitive
- Un sistema di notifiche integrato per tenere gli utenti informati

Sviluppi futuri potrebbero includere:

- Implementazione di meccanismi di caching per migliorare ulteriormente le prestazioni
- Aggiunta di funzionalità per la gestione di promozioni e sconti
- Integrazione con sistemi esterni come servizi di mappatura e pagamenti
- Implementazione di meccanismi di analisi dati per insight commerciali